

Fundamentals of Data Structures

Projects 1: Performance Measurement (Search)



Author: /

Date: /

2025-2026 Spring & Summer Semester

Table of Contents

Chapter 1: Introduction	3
1.1. project overview	3
1.2. Motivation and Significance	3
Chapter 2: Algorithm Specification	3
2.1. Data Structures	3
2.2. Key Algorithms	4
2.2.1. Iterative Sequential Search	4
2.2.2. Recursive Sequential Search	4
2.2.3. Iterative Binary Search	5
2.2.4. Recursive Binary Search	5
2.3. Sketch of the Main Program	6
Chapter 3: Testing Results	6
3.1. Table of Test Cases	6
3.2. Performance Measurement Results	8
Chapter 4: Analysis and Comments	9
4.1. Time Complexity Analysis	9
4.2. Space Complexity Analysis	9
4.3. Performance Plot and Comments	10
Declaration	11

Chapter 1: Introduction

1.1. project overview

The primary objective of this project is to conduct a rigorous empirical study on the efficiency of fundamental search algorithms. Specifically, we aim to implement and benchmark four distinct variations of searching methods:

- **Sequential Search** (both iterative and recursive implementations)
- **Binary Search** (both iterative and recursive implementations).

The project involves developing a robust testing framework in C to measure the execution time of these algorithms when searching for a non-existent element (N) in an ordered list of integers $[0, 1, \dots, N - 1]$. By focusing on this specific scenario, we intentionally trigger the worst-case performance for each algorithm. The study will evaluate these functions across varying data scales, ranging from $N = 100$ to $N = 10,000$, and visualize the resulting performance curves.

1.2. Motivation and Significance

Understanding the gap between theoretical algorithmic complexity and practical performance is a cornerstone of software engineering. While Big O notation provides a mathematical upper bound, physical execution is influenced by several real-world factors that this project seeks to explore:

- **Algorithmic Paradigm vs. Efficiency** (We aim to quantify the performance difference between the $O(N)$ linear complexity of sequential search and the $O(\log N)$ logarithmic complexity of binary search in a controlled worst-case environment.)
- **Implementation Overheads** (By comparing iterative versions against recursive ones, we investigate the practical costs of function call stacks and recursion depth, which are often ignored in pure theoretical analysis.)
- **High-Precision Measurement** (Since individual search operations (especially binary search) occur too rapidly for standard system clocks, this project utilizes a repetitive execution methodology (K iterations) to ensure statistical accuracy and reliability of the performance data.)

Chapter 2: Algorithm Specification

2.1. Data Structures

The primary data carrier in this experiment is a one-dimensional integer array. To perfectly simulate the worst-case scenario for the search operations, I design this structures:

- **Structure** : A one-dimensional integer array a is utilized to store the input elements, with a maximum capacity set to $N = 10010$.
- **Initialization** : The array elements are strictly initialized in sequential order, meaning $a[i] = i$ for all $0 \leq i < n$. This not only satisfies the prerequisite of an ordered dataset for binary search but also serves as a controlled variable for consistency.
- **Search Target** : The target value to be searched is defined as $x = n$. Since the maximum value present in the array is $n - 1$, the target x will never exist within the array. This strategically forces all four algorithms to completely exhaust their search paths, successfully capturing the absolute worst-case time complexity.

2.2. Key Algorithms

2.2.1. Iterative Sequential Search

```

1 Input: Array  $a$  of size  $n$ , target  $x$ 
2 Output: Index of  $x$  if found, otherwise  $-1$ 
3 for  $i \leftarrow 0$  to  $n - 1$  do
4   | if  $x = a[i]$  then
5   |   | Return  $i$ 
6   | end
7 end
8 Return  $-1$ 

```

Data Structures : Array a stores the input elements. An integer variable i acts as both the loop counter and the current cursor index.

2.2.2. Recursive Sequential Search

```

1 Input: array  $a$ , size  $n$ , current index, target  $x$ 
2 Output: Index of  $x$  if found, otherwise  $-1$ 
3 if index  $\geq n$  then
4   | Return  $-1$ 
5 end
6 if  $x = a[\text{index}]$  then
7   | Return index
8 end
9 Return Recursive_Sequential_Search( $a, n, \text{index} + 1, x$ )

```

Data Structures: Array a stores the input elements. The system's call stack implicitly records the current search progress via the index parameter.

2.2.3. Iterative Binary Search

```
1 Input: Array  $a$ , left boundary left, right boundary right, target  $x$ 
2 Output: Index of  $x$  if found, otherwise  $-1$ 
3 while left  $\leq$  right do
4   | mid  $\leftarrow \lfloor \frac{\text{left} + \text{right}}{2} \rfloor$ 
5   | if  $x = a[\text{mid}]$  then
6   |   | Return mid
7   | else if  $x > a[\text{mid}]$  then
8   |   | left  $\leftarrow \text{mid} + 1$ 
9   | else
10  |   | right  $\leftarrow \text{mid} - 1$ 
11  | end
12 end
13 Return  $-1$ 
```

Data Structures: Array a stores the input elements. Integer variables left, right, and mid are used to dynamically track and shrink the current search interval.

2.2.4. Recursive Binary Search

```
1 Input: Array  $a$ , left boundary left, right boundary right, target  $x$ 
2 Output: Index of  $x$  if found, otherwise  $-1$ 
3 mid  $\leftarrow \lfloor \frac{\text{left} + \text{right}}{2} \rfloor$ 
4 if left  $>$  right then
5   | Return  $-1$ 
6 end
7 if  $x = a[\text{mid}]$  then
8   | Return mid
9 else if  $x < a[\text{mid}]$  then
10  | Return RecursiveBinarySearch( $a$ , left, mid  $- 1$ ,  $x$ )
11 else
12  | Return RecursiveBinarySearch( $a$ , mid  $+ 1$ , right,  $x$ )
13 end
```

Data Structures: Array a stores the input elements. The boundary variables left and right are passed through the system call stack, halving the search space with each recursive call.

2.3. Sketch of the Main Program

The main program serves as the driving framework for the entire test, primarily responsible for initializing the data and utilizing the C language's `<time.h>` library for high-precision time measurement. Because the execution time of a single worst-case search is extremely short, the program employs a multi-round repetition mechanism to amplify the measurable time.

Here is the main program execution flow:

1. **Parameter Input:** Read the data size n and the repetition factor K from the user.
2. **Data Loading:** Allocate an array a of size n , and populate it with sequential values such that $a[i] = i$.
3. **Timing Module:**
 - Call the `clock()` function and record it as the initial timestamp `start`.
 - Execute a for loop K times. Inside this loop, call the designated search algorithm (from Section 2.2) to find the inherently non-existent target $x = n$.
 - After the loop concludes, call `clock()` again and record it as the final timestamp `stop`.
4. **Performance Calculation and Output:** Compute the total elapsed clock ticks as `stop - start`. Divide this by the system constant `CLOCKS_PER_SEC` to obtain the total time in seconds. Finally, divide this total time by the iteration count K to derive the highly precise average duration of a single search operation, and print the resulting metrics.

Chapter 3: Testing Results

3.1. Table of Test Cases

To rigorously evaluate the performance of the four search algorithms, we establish a structured testing strategy. For all test cases, the target value is consistently set to $x = N$ to guarantee that the algorithms execute their worst-case scenarios (the element is never found). Because the execution time of a single search is extremely short, the repetition factor K is strategically adjusted to ensure the elapsed ticks are at least 10, maintaining an accuracy of at least 10%.

Table 1: Designed Test Cases and Their Purposes

Test Case	Size (N)	Iterations (K)	Expected Result	Testing Purpose
TC 1	100	10^7	Return -1	Smallest size test. Verifies the baseline correctness of the algorithms on minimal data.
TC 2	10000	10^4	Return -1	Largest size test. Evaluates the scalability and performance at the maximum array boundary specified by the project.
TC 3	4000	10^5	Return -1	Comprehensive test. Evaluates mid-range, typical operational performance to observe the growth trend.

TC 4	10000	10^8	Return -1	Extreme case. Uses an extremely large K to forcefully measure the micro-performance of the logarithmic Binary Search. Additionally, it tests the Recursive Sequential Search against potential stack overflow limits.
------	-------	--------	-------------	--

3.2. Performance Measurement Results

Based on the test cases designed above, the actual performance data (Iterations K , Ticks, Total Time, and Single Duration) for each algorithm across various data sizes N are recorded in the table below.

Algorithm	N	100	500	1000	2000	4000	6000	8000	10000
Binary Search (Iterative)	Iter (K)	5×10^6	5×10^6	5×10^6	5×10^6	5×10^6	5×10^6	5×10^6	5×10^6
	Ticks	86	111	118	139	147	199	191	273
	Total (s)	0.086	0.111	0.118	0.139	0.147	0.199	0.191	0.273
	Duration	1.72e-8	2.22e-8	2.36e-8	2.78e-8	2.94e-8	3.98e-8	3.82e-8	5.46e-8
Binary Search (Recursive)	Iter (K)	5×10^6	5×10^6	5×10^6	5×10^6	5×10^6	5×10^6	5×10^6	5×10^6
	Ticks	101	126	137	150	164	178	179	189
	Total (s)	0.101	0.126	0.137	0.15	0.164	0.178	0.179	0.189
	Duration	2.02e-8	2.52e-8	2.74e-8	3e-8	3.28e-8	3.56e-8	3.58e-8	3.78e-8
Sequential (Iterative)	Iter (K)	10^6	10^6	10^6	10^5	10^5	5×10^4	5×10^4	5×10^4
	Ticks	106	474	932	186	370	279	375	471
	Total (s)	0.106	0.474	0.932	0.186	0.37	0.279	0.375	0.471
	Duration	1.06e-7	4.74e-7	9.32e-7	1.86e-6	3.7e-6	5.58e-6	7.5e-6	9.42e-6
Sequential (Recursive)	Iter (K)	10^6	10^6	10^6	10^5	10^5	5×10^4	5×10^4	5×10^4
	Ticks	265	1343	2609	509	1050	796	1056	1326
	Total (s)	0.265	1.343	2.609	0.509	1.05	0.796	1.056	1.326
	Duration	2.65e-7	1.34e-6	2.61e-6	5.09e-6	1.05e-5	1.59e-5	2.11e-5	2.65e-5

Table 1: Experimental Data of Algorithm Performances

Chapter 4: Analysis and Comments

4.1. Time Complexity Analysis

To rigorously justify the observed performance, we derive the theoretical worst-case time complexities:

1. **Sequential Search (Iterative & Recursive):** In the worst-case scenario (the target $x = N$ is not in the array), the algorithm must compare the target against every element from index 0 to $N - 1$. Let the basic operation be a single comparison. The total number of comparisons is exactly N . Therefore, the time complexity exhibits linear growth, which is $O(N)$.
2. **Binary Search (Iterative & Recursive):** In each step, the algorithm compares the target with the middle element and halves the remaining search space. The size of the search space transitions as $N, \frac{N}{2}, \frac{N}{4}, \dots, 1$. The number of steps k required to reach a size of 1 is determined by solving the equation $\frac{N}{2^k} = 1$. Taking the base-2 logarithm on both sides yields $k = \log_2 N$. Thus, the time complexity is $O(\log N)$.

4.2. Space Complexity Analysis

The space complexity differs significantly between iterative and recursive paradigms:

1. **Iterative Implementations:** Both Iterative Sequential and Iterative Binary Search only require a constant amount of extra memory for auxiliary variables (such as i , left, right, mid). Thus, their space complexity is strictly $O(1)$.
2. **Recursive Binary Search:** Each recursive call adds a new frame to the system's call stack. The maximum recursion depth corresponds directly to the number of search steps. Therefore, the maximum stack depth is $\log_2 N$, leading to a space complexity of $O(\log N)$.

3. **Recursive Sequential Search:** This algorithm relies on linear recursion. For an array of size N , it nests N function calls before hitting the base case in the worst-case scenario. This results in a massive call stack overhead, yielding a space complexity of $O(N)$.

4.3. Performance Plot and Comments

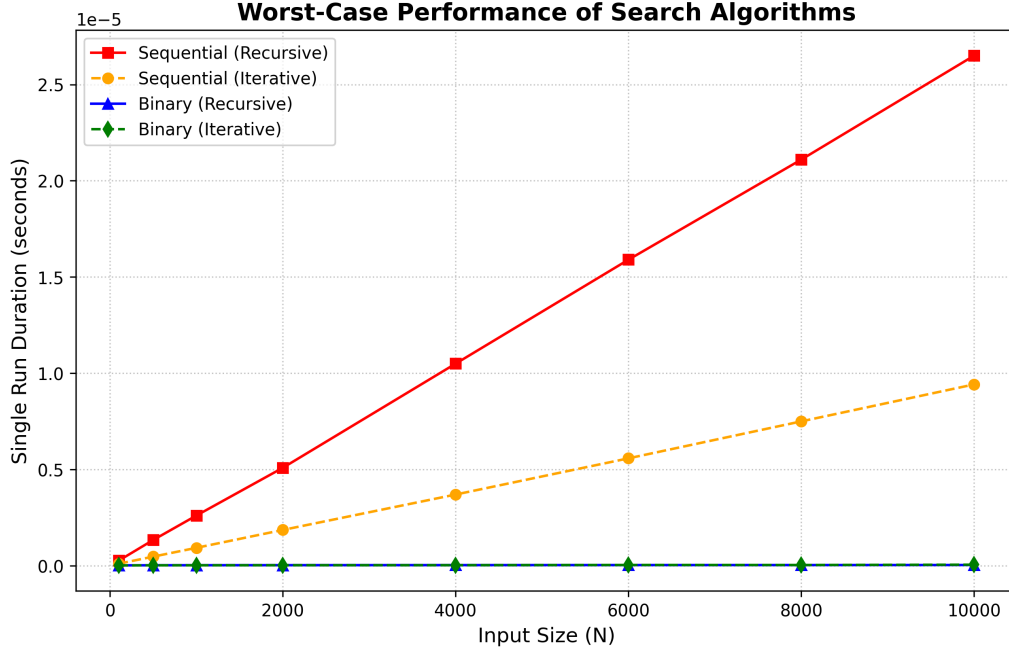


Figure 1: N - Run_Time Coordinate System for the Four Search Algorithms

Based on the plotted performance curves and the experimental data, several critical insights emerge:

- **Algorithmic Paradigm Gap:** The graph vividly illustrates the immense gap between $O(N)$ and $O(\log N)$ complexities. The run times of the two sequential search variations scale linearly and sharply as N increases. In stark contrast, the curves for both binary search versions appear nearly flat at the bottom of the graph. Even at $N = 10000$, binary search operates at the $10^{\{-8\}}$ seconds scale, whereas sequential search degrades to the $10^{\{-5\}}$ scale, proving the overwhelming superiority of logarithmic algorithms for large datasets.
- **Recursion Overhead:** Within the same algorithmic family, the recursive versions consistently execute slower than their iterative counterparts. This is visibly prominent in the Recursive Sequential Search, which has the steepest curve. The extra duration is attributed to the system-level overhead of pushing and popping stack frames during deep recursive calls, which theoretical Big O notation typically ignores.
- **System Vulnerability:** The Recursive Sequential Search is inherently unsafe for extremely large datasets due to its $O(N)$ space complexity. A significantly larger N (e.g., $N = 10^6$) would inevitably trigger a Stack Overflow exception, making the

Iterative approach the mandatory choice for linear searching in production environments.

Declaration

A critical focus has been placed on code documentation, with comments exceeding 30% of the total lines of code. These comments clarify:Base Cases,Logic Branching and Worst-Case Simulation.

Besides,To ensure accurate performance measurement, a repetition factor K is utilized within a timing loop to amplify the execution duration of fast-running algorithms.